Email Spam Detection Web App

1. Aim

To design and develop a web application that classifies an email (or message) as **Spam** or **Not Spam (Ham)** using Machine Learning and Natural Language Processing techniques. The user pastes the email text into a web page, and the system instantly predicts the class along with a confidence score.

Objectives

- Collect and explore a labelled spam/ham dataset.
- Clean and normalise the raw text using NLP preprocessing.
- Convert text into numerical features using TF-IDF.
- Train a Multinomial Naive Bayes classifier and evaluate its performance.
- Deploy the trained model as a Flask web application.

2. Introduction

Spam emails waste time, consume bandwidth, and are often used for phishing and fraud. Manual filtering is not practical, so automatic text classification is used. This project uses a supervised learning approach: the model learns word patterns from labelled examples (for instance, words such as *free*, *win*, *prize*, *urgent* appear far more often in spam) and then predicts the class of new, unseen emails.

3. Technologies Used

Component	Tool
Language	Python 3
Data handling	Pandas, NumPy
NLP	NLTK (stopwords, Porter Stemmer)
ML	Scikit-learn (TF-IDF, Multinomial Naive Bayes)
Web framework	Flask

Component	Tool
Frontend	HTML, CSS
Model storage	Pickle

4. System Architecture

The flow of the system is:

1. User enters email text in the web page.
2. Flask receives the text through a POST request.
3. Text is preprocessed (lowercase, remove punctuation, remove stopwords, stemming).
4. The saved TF-IDF vectorizer converts the text into a numeric vector.
5. The saved Naive Bayes model predicts Spam or Ham with a probability.
6. The result is displayed on the web page.

5. Algorithm

Algorithm: Email Spam Detection using TF-IDF and Multinomial Naive Bayes

Training phase

1. Load the labelled dataset (label, message).
2. Encode labels: ham = 0, spam = 1.
3. For every message: convert to lowercase, remove punctuation and numbers, tokenize, remove stopwords, apply stemming, and join tokens back into a string.
4. Split the data into 80% training and 20% testing (stratified).
5. Compute TF-IDF features: $TF\text{-}IDF(t, d) = TF(t, d) \times \log(N / DF(t))$.
6. Train Multinomial Naive Bayes using Bayes' theorem: $P(class \mid words) \propto P(class) \times \Pi P(word \mid class)$.
7. Evaluate on the test set (accuracy, precision, recall, F1, confusion matrix).
8. Save the vectorizer and model to disk.

Prediction phase

1. Receive the email text from the web form.
2. Apply the same preprocessing steps as in training.

3. Transform the text using the saved TF-IDF vectorizer.
4. Compute class probabilities with the saved model.
5. Output **Spam** if P(spam) > P(ham), otherwise **Not Spam**, and show the confidence.

6. Module-wise Implementation

Install the requirements first:

```
pip install pandas numpy scikit-learn nltk flask
```

Module 1: Data Collection and Exploration

Purpose: Load the dataset and understand its size and class balance.

```
import pandas as pd

df = pd.read_csv("spam.csv", encoding="latin-1")[["v1", "v2"]]
df.columns = ["label", "message"]
df["label_num"] = df["label"].map({"ham": 0, "spam": 1})

print("Shape:", df.shape)
print(df["label"].value_counts())
print(df.head())
```

Output:

```
Shape: (5572, 3)
ham     4825
spam     747
Name: label, dtype: int64

  label                                            message  label_num
0   ham  Go until jurong point, crazy.. Available only ...          0
1   ham                      Ok lar... Joking wif u oni...          0
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...          1
3   ham  U dun say so early hor... U c already then say...          0
4   ham  Nah I don't think he goes to usf, he lives aro...          0
```

Observation: The dataset has 5,572 messages, of which about 13.4% are spam, so the data is imbalanced.

Module 2: Text Preprocessing

Purpose: Clean the text so that the model focuses on meaningful words.

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

nltk.download("stopwords")
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    words = [stemmer.stem(w) for w in text.split() if w not in stop_words]
    return " ".join(words)

df["clean"] = df["message"].apply(preprocess)

print("Original :", df["message"][2])
print("Cleaned  :", df["clean"][2])
```

Output:

```
Original : Free entry in 2 a wkly comp to win FA Cup final tkts 21st May
2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply
08452810075over18's
Cleaned  : free entri 2 wkli comp win fa cup final tkt 21st may 2005 text fa
87121 receiv entri question std txt rate c appli 08452810075over18
```

Module 3: Feature Extraction and Data Splitting

Purpose: Convert text into numeric TF-IDF vectors and split into training and testing sets.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X_train_txt, X_test_txt, y_train, y_test = train_test_split(
    df["clean"], df["label_num"],
    test_size=0.2, random_state=42, stratify=df["label_num"]
)

vectorizer = TfidfVectorizer(max_features=3000)
X_train = vectorizer.fit_transform(X_train_txt)
X_test = vectorizer.transform(X_test_txt)

print("Training matrix:", X_train.shape)
print("Testing matrix :", X_test.shape)
```

Output:

```
Training matrix: (4457, 3000)
Testing matrix : (1115, 3000)
```

Module 4: Model Training

Purpose: Train the Multinomial Naive Bayes classifier and save it for the web app.

```
import pickle
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB()
model.fit(X_train, y_train)

pickle.dump(model, open("model.pkl", "wb"))
pickle.dump(vectorizer, open("vectorizer.pkl", "wb"))
print("Model trained and saved successfully.")
```

Output:

```
Model trained and saved successfully.
```

Module 5: Model Evaluation

Purpose: Measure how well the model performs on unseen data.

```
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

y_pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=["Ham", "Spam"]))
```

Output:

```
Accuracy: 0.9695

Confusion Matrix:
 [[965   0]
 [ 34 116]]

Classification Report:
              precision    recall  f1-score   support

         Ham       0.97      1.00      0.98       965
        Spam       1.00      0.77      0.87       150

    accuracy                           0.97      1115
   macro avg       0.98      0.89      0.93      1115
weighted avg       0.97      0.97      0.97      1115
```

Observation: No genuine email was wrongly marked as spam (spam precision is 1.00), which is the most important property for a spam filter. Some spam messages (34 of 150) were missed, which can be improved by balancing the data or tuning the model.

Module 6: Flask Backend

Purpose: Load the saved model and expose a prediction route.

```
# app.py
import re
import pickle
import nltk
from flask import Flask, render_template, request
from nltk.corpus import stopwords
```

```
from nltk.stem import PorterStemmer

nltk.download("stopwords")
stop_words = set(stopwords.words("english"))
stemmer = PorterStemmer()

model = pickle.load(open("model.pkl", "rb"))
vectorizer = pickle.load(open("vectorizer.pkl", "rb"))

app = Flask(__name__)

def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    words = [stemmer.stem(w) for w in text.split() if w not in stop_words]
    return " ".join(words)

@app.route("/", methods=["GET", "POST"])
def home():
    result, confidence, email = None, None, ""
    if request.method == "POST":
        email = request.form["email"]
        vec = vectorizer.transform([preprocess(email)])
        pred = model.predict(vec)[0]
        prob = model.predict_proba(vec)[0]
        result = "Spam" if pred == 1 else "Not Spam"
        confidence = round(max(prob) * 100, 2)
    return render_template("index.html", result=result,
                           confidence=confidence, email=email)

if __name__ == "__main__":
    app.run(debug=True)
```

Output (terminal):

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Module 7: Frontend (User Interface)

Purpose: Provide a simple page where the user enters the email and sees the result.

```
<!-- templates/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Email Spam Detector</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f2f4f8; display:
flex; justify-content: center; padding: 40px; }
    .box { background: #fff; padding: 30px; border-radius: 10px; width:
520px; box-shadow: 0 2px 10px rgba(0,0,0,.1); }
    textarea { width: 100%; height: 160px; padding: 10px; font-size: 15px; }
    button { margin-top: 12px; padding: 10px 24px; background: #2563eb;
color: #fff; border: 0; border-radius: 6px; font-size: 15px; cursor: pointer;
}
    .spam { color: #b91c1c; font-weight: bold; font-size: 20px; }
    .ham  { color: #15803d; font-weight: bold; font-size: 20px; }
  </style>
</head>
<body>
  <div class="box">
    <h2>Email Spam Detector</h2>
    <form method="POST">
      <textarea name="email" placeholder="Paste the email text here..."
required>{{ email }}</textarea>
      <button type="submit">Check</button>
    </form>
    {% if result %}
      <p class="{{ 'spam' if result == 'Spam' else 'ham' }}">
        Result: {{ result }} ({{ confidence }}% confidence)
      </p>
    {% endif %}
  </div>
</body>
</html>
```

7. Testing the Web Application

Run the application with `python app.py` and open `http://127.0.0.1:5000` in a browser.

Test	Input email	Output
1	"Congratulations! You have won a free prize of $1000. Click here to claim now."	Spam (about 95% confidence)
2	"URGENT! Your account is suspended. Call 09061701461 to claim your reward."	Spam (about 97% confidence)
3	"Hi team, the meeting is shifted to 3 PM tomorrow. Please confirm."	Not Spam (about 98% confidence)
4	"Can you send me the project notes before tonight?"	Not Spam (about 97% confidence)

Confidence values are indicative and will differ slightly on your trained model.

8. Final Result

Parameter	Value
Dataset size	5,572 messages (4,825 ham, 747 spam)
Train / test split	80% / 20%
Features	TF-IDF, 3,000 terms
Algorithm	Multinomial Naive Bayes
Accuracy	about 96.9%
Spam precision	1.00
Spam recall	0.77
Spam F1-score	0.87
Deployment	Flask web application

The system successfully classifies user-entered emails as Spam or Not Spam through a web interface, with high accuracy and no false alarms on the test set.

9. Conclusion

The Email Spam Detection Web App meets its aim. Using NLP preprocessing, TF-IDF feature extraction, and the Naive Bayes algorithm, it detects spam with about 97% accuracy and presents the result through a simple, user-friendly web page. Naive Bayes was chosen because it is fast, needs little training data, and works very well for text classification.

10. Future Scope

- Handle class imbalance (SMOTE or class weights) to improve spam recall.
- Compare with Logistic Regression, SVM, Random Forest, and deep learning models such as LSTM or BERT.
- Add email header and URL analysis for better phishing detection.
- Connect to Gmail or Outlook through an API for live inbox filtering.
- Deploy on a cloud platform (Render, AWS, or Heroku) for public access.

11. References

- UCI Machine Learning Repository: SMS Spam Collection Dataset.
- Scikit-learn documentation: scikit-learn.org.
- Flask documentation: flask.palletsprojects.com.
- NLTK documentation: nltk.org.